WelsherLab smtBatchTraj2026 README

Version Notes

Version: 1.0
Last updated: 2026-08-09
This batchTraj code does the basic analysis for smt tracking experiments. For an easier time modifying it to fit your application, please check the Modification guide section.
MATLAB version: 2024a/b
MATLAB code reference: smtBatchTraj2026_V1
LabVIEW VI reference: Galvo Track v21.3 Electroshock

Written by Chan

Main sections:

Prerequisites

This script requires the following add-ons in MATLAB: Data Acquisition Toolbox (for `tdmsread()`), and Parallel Computing Toolbox (for `parfor` loop).

I would recommend about 1GB of free space on your local hard drive to comfortably run this script without running into trouble with the TDMS copy process.

If you plan on using the `notifySlack` feature, please have your channel ID ready. If you don't have one, see Getting channel ID for MATLAB Notifier.

Pipeline explanation

Generally, when you hit Run on the script:

1. A pop-up for data folder selection opened for user to select which folder to execute the code on.

2. Filter TDMS files
3. For each trajectory:
    1. Copy the .tdms and .tdms_index files to a local folder (optional)
    2. Read the TDMS
    3. For each data field, convert raw bits to physical units
    4. Downsample and cut-off the edges
    5. Save raw and filtered trajectory
    6. Calculate MSD
    7. Generate individual plots and optionally, data collages
    8. Add trajectory summary into `tracking`
4. Save `tracking` (now contains information of all processed trajectory) as `tracking.mat`
5. Organization and clean-up
6. Notify user that code has finished running (optional)

Expected output

Inside the folder that the user selected in the pop-up dialog, a folder named "Processed Data" is created to store all outputs of this code: `tracking.mat`, "Data Collages" folder (optional), and folders for individual trajectory.

For each trajectory, a folder with its name is generated. The folder contains:

By default	Optional
- 3dMSD.fig - 3dtraj.fig - galvoX.fig - galvoY.fig - intensity.fig - stageX.fig - stageY.fig - stageZ.fig - x_k.fig - y_k.fig - z_k.fig - (trajectory name).mat	- collage.fig - msd1D_x.fig - msd1D_y.fig - msd1D_z.fig

`(trajectory name).mat` (e.g., TR001.mat) contains the raw `trackData` and the filtered/downsampled `trackDataFilt` matrices of a particular trajectory.

If data collages are also generated as PNGs, the .png files are all stored in "Data Collages" folder.

Params settings

At the top of the script is the Params section which control how the batch analysis behaves.

File paths

The file path for `masterDataFolderPath` is wherever you want the folder selection pop-up to open to. When the script starts, `uigetdir()` opens from this location and asks the user to select the experiment folder to analyze.

`localDataFolderPath` is the path to a local folder on your computer. In case you opt to read from a local TDMS copy (faster performance for non-lab PCs), this is where the TDMS files are copied to and where everything from Expected output is stored before it is optionally moved onto the network.

> **Note**
>
> Please make sure that you have unrestricted read/write permission for wherever you pick for `localDataFolderPath`.

scriptSettings

This variable controls the workflow.

`createDupFolderIfExist` determines what happened if, within "Processed Data" folder, a folder with the exact trajectory name already exists. If set to false, it will overwrite whatever files inside the trajectory folder. If set to true, it will create a duplicate folder to store it. For example, a folder named "TR001" already exists, so data will be stored to "TR001 (1)".

`userParallelComputing` and `numParallelWorkers` are settings related to parallel computing. When you set the former to true, the scripts will initialize a parallel pool of `numParallelWorkers` workers and run a `parfor` loop to process the trajectory. If it is set to false, a regular `for` loop is run.

> **About parallel computing**
>
> If you haven't initialized the parallel pool before running the script, the initialization process will ad a few mins to the overall processing time. This only needs to be once per MATLAB session (unless it times out for being idle).
>
> Depending on your computer, you may or may not benefit from parallel computing. I'd recommend playing around with `numParallelWorkers` to find the most optimal value for your case.
>
> I personally didn't find much improvement when analyzing the trajectory in parallel, so I just leave `useParallelComputing` to false.

`useLocalTDMSCopy` is recommended for any non-lab PCs. When this is true, each network TDMS file is first copied to a temporary location at the `localDataFolderPath` location. The script will then use `tdmsread()` on this local file. To visualize the performance differences with this setting, check Performance benchmark.

If analyzed data was stored locally, `moveLocalFoldertoNetwork` will determine if those results in the local "Processed Data" folder will be moved back to the network location (where the original TDMS files are located) after analysis.

`notifySlack` is pretty straightforward. If you want a Slack notification after the script is done, set this to true. Just make sure that `matlabNotifierID` corresponds to your channel ID. Again, for how to set this up, refer to Getting channel ID for MATLAB Notifier.

As the names suggest, `makeDataCollage` and `saveCollagePNG` are for the collages. A data collage is a figure with XYZ, MSD (3D and 1D if enabled), intensity, and the 3D trajectory all laid out in one place. If `saveCollagePNG` is true, the collage will be saved as a .png file also. The .fig collage file is stored in the individual trajectory folder while that .png is stored in "Data Collages" folder.

`stageTracking` changes the trajectory processing slightly. Set this to true if you used stage tracking instead of galvo tracking. Details are discussed in Analysis section.

`fileSizeCutoff` and `trajDurationCutoff` are for TR file filtering. Before trajectory analysis begins, any TR files smaller than `fileSizeCutoff` is ignored. Similarly, once the TDMS file is read, `trajDurationCutoff` applies another layer of filter to skip over any trajectories with duration shorter than the threshold value.

`calc1dMSD` controls whether MSD fitting and subsequent calculations are performed on each axis independently. If true, code will generate the corresponding plots and include them in the data collage (if enabled).

`trajMatIsTable` controls the format of the raw trajectory data `trackData` and the decimated `trackDataFilt`. If true, the .mat file of each trajectory saves the variables in table forms which have headers for readability. Otherwise, the array version are saved.

plotSettings

This variable control figure appearances.

These settings are passed into the plotting functions instead of being hard-coded independently for every figure. As such, settings like `lineWidth`, `lineColor`, `fontSize`, and `plotGrid` are applied to all figures generated.

`background` accommodates personal preferences for light- or dark-themed figures. You can either do `plotSettings.background = 'white'` or `plotSettings.background = 'black'`. White background figures will have black text and axes, while the black counterpart has the opposite.

> **Small bug**
>
> For dark-themed collage, the figure title is grey despite the code specifically sets it to white. I spent a bit of time trying to fix it before giving up...

Pre-analysis prep

This section does the prep work before any individual trajectory is processed. Two main objectives in this section is:

1. Create folders to store analysis data
2. Filter the TR files in the experiment data folder

The first goal is straightforward enough. It only looks slightly complex in the code because it needs to take into account whether these folders need to be created in the network folder or the local folder.

For the second goal, the script first obtains a list of all TDMS files in the experiment folder. It then filters the files such that the following files are ignored:

1. Filename contains "IM"
2. File size is smaller than `scriptSettings.fileSizeCutoff` value

The number of trajectories to processed are stored in `nFiles`. `trajectoryNames` has the corresponding names. `processedDataSavePaths` is the save location for analyzed data.

`reservedSavePaths` is particular to parallel processing. This array prevents two trajectories in the current batch from accidentally selecting the same output directory before either folder has actually been created.

Analysis

As mentioned before, you can opt to process the trajectory using parallel computing or typical sequential computing. In the case of parallel computing, the parallel pool is initialized and analysis is run with a `parfor` loop.

Either way, the main analysis function `processTrajectory()` is called. Most of the actual work performed on any singular trajectory happens here.

`processTrajectory()` : Processing TDMS files

Note that this function adds three new fields to `scriptSettings`:

1. `aciveFeedbackLoopPeriod`
2. `downSamplingFactor`
3. `downSamplingEdgeTrim`

My default value for `activeFeedbackLoopPeriod` is 10e-6, meaning the sampling interval for raw trajectory is 10 μs (100 kHZ). This value is subsequently used to calculate trajectory duration, raw time vector, and the downsampled sampling interval.

To read the TDMS files, the code considers where to read said files. If you are reading directly from the network, a simple `tdmsread()` is called. If you opt to read from a local copy, `readTDMSLocalCopy()` is called. This function handles the copying of .tdms and .tdms_index from network to local folder (`localDataFolderPath`), directs the code to read from this local copy, and deletes said copy when done reading.

Once data is pulled out from the TDMS file, the trajectory duration is calculated based on the number of data points in the TDMS data and the raw sampling interval `scriptSettings.activeFeedbackLoopPeriod`. If the duration is less than `scriptSettings.trajDurationCutoff`, the rest of the analysis function is skipped.

If the trajectory is sufficiently long, we begin processing (for real this time!). The relevant data fields from the TDMS file (i.e., intensity, stage XYZ, galvo XY, etc.) are converted from bits into physical units (μm and nm).

When `scriptSettings.stageTracking = true`, the trajectory is processed with the same pipeline as the original trajLoadTDMSCalibrated. In this case, galvo X and Y are converted slightly differently (using `BITSCONVERSION_X` and `BITSCONVERSION_Y`), and the stage X and Y values are updated based on some galvo calibration value.
When `scriptSettings.stageTracking = false`, the trajectory is processed with the same pipeline as the original trajLoadTDMS_Galvo. The bits values for galvo X and Y are simply converted using `BITSCONVERSION_GALVO`.

A decimated data set `TrackDataFilt` is generated. This is controlled by `scriptSettings.downSamplingFactor` and `scriptSettings.downSamplingEdgeTrim`. To be consistent with the original batchtraj_tdms_Galvo, the `downSamplingFactor` is set to 100, bringing the temporal resolution for the filtered data set to 1 ms after `decimateColumns()`. Meanwhile `downSamplingEdgeTrim` is set to 100 to remove the first and last 100 data points of the new data set. This remove filter-edge transients introduced by downsampling.

Here a .mat file with the trajectory name is generated. Inside are the full-resolution raw data `trackData` and the downsampled `trackDataFilt`. The format of these variables (array or table) is decided by `scriptSettings.trajMatIsTable`.

> **Note**
>
> All further calculations and plotting only uses the filtered/downsampled data, not the raw data.

`processTrajectory()` : Calculating MSD

This script calculates and pulls information from the 3D MSD by default. 1D MSD information are considered extra and is controlled by `scriptSettings.calc1dMSD`.

> **Note**
>
> The naming of `scriptSettings.calc1dMSD` is a slight misnomer since the 1D MSD is calculated regardless to get the 3D MSD. That said, if this setting is false, the code would skip over the MSD fitting and subsequent calculations for the 1D version.

When calling the main MSD function `calculateMSDXYZ()`, the code first determines what the XYZ input matrix ought to be based on `scriptSettings.stageTracking`. If it's true, the script uses `[xStage, yStage, zStage]`, otherwise `[xGalvo, yGalvo, zStage]` is used.

The maximum lag `maxLag` is limited to 1/4 of the number of points in the trajectory.

`meanSquaredDisplacementFFT()` calculates the 1D MSD for X, Y, Z. For a detailed explanation of this function, see MSD calculation via autocorrelation. The 3D MSD is obtained by summing the 1D MSDs together.

For diffusion coefficient (D) of random motion, the slope (part of `calculateMSDXYZ()`) and R2 of a linear MSD fit (`calcR2()`) are calculated. The slope calculation ignores the y-intercept, basically assuming a zero y-intercept.

The hydrodynamic radius (r) is calculated (`diffusionToRadius()`) using the Stokes-Einstein equation:

$$r = \frac{k_B T}{6\pi\eta D}$$

where k_B is the Boltzmann constant, T is temperature, and η is the viscosity of the solvent (water by default).

`processTrajectory()`: Plotting

> **Note**
>
> The plotting process in this script leverages a little trick to speed things up. Since this is helpful beyond the scope of this script, it has its own tiny section!

Although `plotSettings` is an input argument, the actual plot settings fed into the plotting functions are actually from `plotSettingsLocal`. It is basically just a copy of `plotSettings` but with additional fields specific to individual trajectory (i.e., trajectory name for plot title and the save path for .fig files).

`plotTrackData()` is the central plotting function for individual plots. To determine which plot to generate, it reads input argument `plotType`. Two types of plots are generated: those with Time on the x-axis (`plotVersusTime()`) and MSD plots (`plotMSD()`). Each `plotType` decides the corresponding XY

values, applies the correct axes labels, handles appearance tweaks from `plotSettings` via `applyPlotSettings()`, and saves the figure with `applySaveSettings()`.

For plotting MSD, two variables `msd1DPlotData` and `msd3DPlotData` are used. Both are outputs from `calculateMSDXYZ()` and contain data for XY axes, errorbars, slope of the MSD fit, R2, and particle information (diffusion coefficient and hydrodynamic radius).

It's important to note that the MSD plots do not show all available MSD versus lag points. If there are more than `maxMSDPlotPoints` (currently set to 400), `selectMSDPlotLags()` will limit the displays to `maxMSDPlotPoints` log-spaced lag indices. This eases the workload for `calculateMSDUncertainty()` since it only needs to calculate the error bars for this subset instead of every lag points. This is purely visual-related. The MSD fit, R2, and particle information are still calculated using the full MSD dataset.

If `scriptSettings.makeDataCollage = true`, the script runs `createTrajectoryCollage()`. The collage layout depends on `scriptSettings.stageTracking` and `scriptSettings.calc1dMSD`, as defined by `defaultCollageTiles()`.

Post-analysis

Now that analysis for all trajectories are done, the script runs through some clean-up and organization code.

Basically,

1. Remove any remaining .tdms and .tdms_index files inside `localDataFolderPath`
2. Save `tracking` variable (containing info for all trajectories) as `tracking.mat`
3. If `scriptSetings.saveCollagePNG = true`, organize collage PNGs into "Processed Data>Data Collages"
4. If `scriptSetings.moveLocalFoldertoNetwork = true`, move processed trajectory folders from the local folder back into the experiment's network folder location. `transferStatuses` holds a record of whether the code runs into any error during the file moving process
5. If `scriptSettings.notifySlack = true`, shoot a message to Slack via `notifySlackWhenDone()`

Modification guide

If you are using a different LabVIEW VI

Different VIs can have different structure for the TDMS output. If you are using a VI other than Galvo Track v21.3 Electroshock, there are a few things I would recommend you look over to ensure that it fits the VI you are using.

In `processTrajectory()`, `tdmsread()` is used for reading the TDMS file. The output of `tdmsread()` is basically a table with headers for the TDMS channels from LABVIEW. When converting bits to physical units, the script utilizes the table headers to get the corresponding data column.

This script hard-codes the TDMS table headers to avoid slowdown with looking up the TDMS channel repeatedly. As such, if your VI has different names for the TDMS channels or if the channels used in the script don't exist in your VI, the code will crash at `processTrajectory()`.

To avoid this, either use TDMS File Viewer in LABVIEW or use the following code in MATLAB

```
tdmsData = tdmsread('PATH TO YOUR TDMS FILE'); %give cell array of table
tdmsData = tdmsData{1}; % get table of the 1st cell
tdmsChannels = tdmsData.Properties.VariableNames;
```

to check that your TDMS channels match what were used in `processTrajectory()`.

Beside that, check that `scriptSettings.activeFeedbackLoopPeriod` value is correct for your VI.

Analysis pipeline mod

The variables `scriptSettings` and `plotSettings` (or `plotSettingsLocal`) are passed to many major functions. Modification is thus made easy because you can add new controls by expanding the structure variable with new fields.

For example, if you need to process the trajectory with a setup that uses voltage value, you can add something like `scriptSettings.voltageUsed` at the top of the script. Then go inside `processTrajectory()` to grab that value for your data processing.

On the topic of `processTrajectory()`, after `trackDataFilt` is generated, the script frequently calls `trackDataColFiltKey()` to pull out specific columns inside `trackDataFilt` based on `plotType` for plotting. The column index are based on how the arrays are ordered in `combinedData`, which in turn affects the column order of `combinedDataFilt` and `trackDataFilt`. If you rearrange or add to `combinedData`, please update `trackDataColFiltKey()`.

Plotting mod

Single plots

If you want to generate additional figures, your figure generation code should be inside `processTrajectory()` after `trackDataFilt` is created. `applyPlotSettings()` is the function to handle the light-/dark- theme switching. `applySaveSettings()` is the function for .fig file saving.

If you want to follow the code organization style of the script, `plotTrackData()` is the main function to modify. Since it calls on `trackDataColFiltKey()`, please make sure that the column indices are correct for your code. You can then add a new `plotType` in `plotTrackData()` and write your new figure generation code inside the switch case block.

Data collages

The main function `createTrajectoryCollage()` adds three new fields to `plotSettings`:

1. `collageRows`
2. `collageCols`
3. `collageTiles`

The first two fields dictate the number of rows and columns for MATLAB `tiledlayout()`. The third field is generated via the initialization function `defaultCollageTiles()`. If you want to adjust the arrangement of specific tiles in the collage, it's the one you want. Also,
you should keep in mind the overall layout from `collageRows` and `collageCols` when modifying the `tile` and `span` fields of the struct in this function.

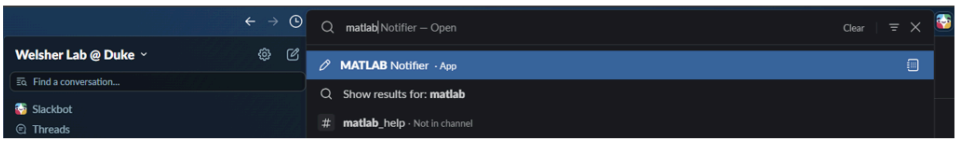
For each tile, `plotTrackData()` is called. Thus, if you want a specific plot, please ensure that the value of the `plotType` fields in `defaultCollageTiles()` matches exactly with the `plotType` in `plotTrackData()`.

Misc.

A section for little nuggets of information

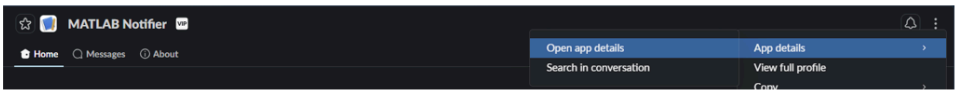
Getting channel ID for MATLAB Notifier

1. Search for MATLAB Notifier in the Slack search bar

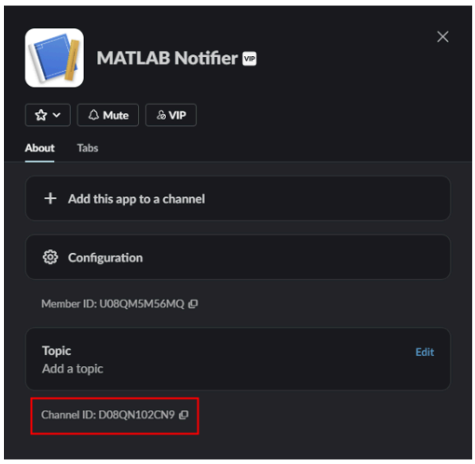


2. Click on MATLAB Notifier. On the right-hand side, click the ⋮ symbol

3. Navigate to App details > Open app details



4. The Channel ID is what the bot uses to identify you



> **Note**
>
> Slack tends to mute/hide notifications after business hours. To enable notifications at all time for this app, set it as a VIP.

Faster processing speed when code generates many figures

When your code generates lots of figure and attempts to show them to the Workspace, it begins to take a toll on processing speed. If you have no need to view every single figure generated as the script is running, opt for

```
fig = figure('Visible', 'off'); % This create an invisible figure
% Add typical code
fig.CreateFcn = 'set(gcbo, ''Visible'', ''on'')'; % This line is important
% Save your figure as usual
```

`CreateFcn` is a callback that MATLAB runs whenever it loads in the data from the .fig file. Essentially, this line says "set the 'Visible' properties to 'on' for this Call Back Object (i.e., the figure that this callback is running on)".

Without the `fig.CreateFcn` line, your figure will open in its invisible state (i.e., a figure would open but the screen is blank). To make it appears, you would have to use `set(gcf, 'Visible', 'on');` for every figure, which gets annoying real fast.

Within this script, this is part of `applySaveSettings()`.

MSD calculation via autocorrelation

Here is a detailed explanation of `meanSquaredDisplacementFFT()`.

1D MSD for lag point index k is defined as

$$MSD(k) = \frac{1}{N-k}\sum_{i=1}^{N-k}(x_{i+k} - x_i)^2$$

where N is the maximum number of lag points `maxLag` (set to be 1/4 of the trajectory length).

A typical implementation for this calculation is

```
for tau= 1:maxLag
    displacement = x(1+tau:end) - x(1:end-tau);
    msd(tau) = mean(displacement.^2);
end
```

It is simple, but scales poorly for longer trajectories as MATLAB has to process more and more lag points.

If we take another look at the MSD equation, particular this part:

$$(x_{i+k} - x_i)^2$$

This can be expanded to

$$(x_{i+k} - x_i)^2 = x_{i+k}^2 - 2x_{i+k}x_i + x_i^2$$

So the full equation can be rewritten as

$$MSD(k) = \frac{1}{N-k}[\sum_{i=1}^{N-k} x_{i+k}^2 - 2\sum_{i=1}^{N-k} x_{i+k}x_i + \sum_{i=1}^{N-k} x_i^2]$$

The middle term has variable x_i and a lagged version of itself x_{i+k}, so we can treat it as an autocorrelation:

$$Corr(k) = \sum_{i=1}^{N-k} x_{i+k}x_i$$

The Wiener-Khinchin theorem states that the power spectral density is the same as the Fourier transform of the autocorrelation function. The power spectral density, in this case, is

$$PSD = FFT(x) \times FFT^*(x) = |FFT(x)|^2$$

where $*$ denotes the complex conjugate.

In other words, we can get $Corr(\tau)$ through

$$Corr(\tau) = real(IFFT(|FFT(x)|^2))$$

In the code, this line

```
nFFT = 2 ^ nextpow2(2 * nSamples);
```

adds zero padding. Without this, FFT in MATLAB treats the signal as if it wraps around (i.e., connecting the end of the trajectory with the beginning). Adding a bunch of zeros at the end neutralizes the effect of the wraparound, effectively giving you the result for linear correlation. (Source: fft - Why should I zero-pad a signal before taking the discrete Fourier transform? - Signal Processing Stack Exchange)

The other two terms in the MSD equation $\sum_{i=1}^{N-k} x_{i+k}^2$ and $\sum_{i=1}^{N-k} x_i^2$ are represented by `sumLate` and `sumEarly`, respectively. Instead of doing repeated sum calculation for every lag, the script does the cumulative sum of squared positions once and use it as a lookup table. `sumLate` and `sumEarly` could then be pulled out through array indexing where the range sum is obtained through index subtraction.

Performance benchmark

> **Tldr;**
>
> Compared to the original batchTraj, this script is 2x faster on lab PC and 10x faster on non-lab PC.

Data folder: \10.236.74.56\data2\Chan\data\260728 smtBatchTraj Test
Ran on newBatchTrajV9.m
Showed in Ultragroup meeting on 2026-07-29

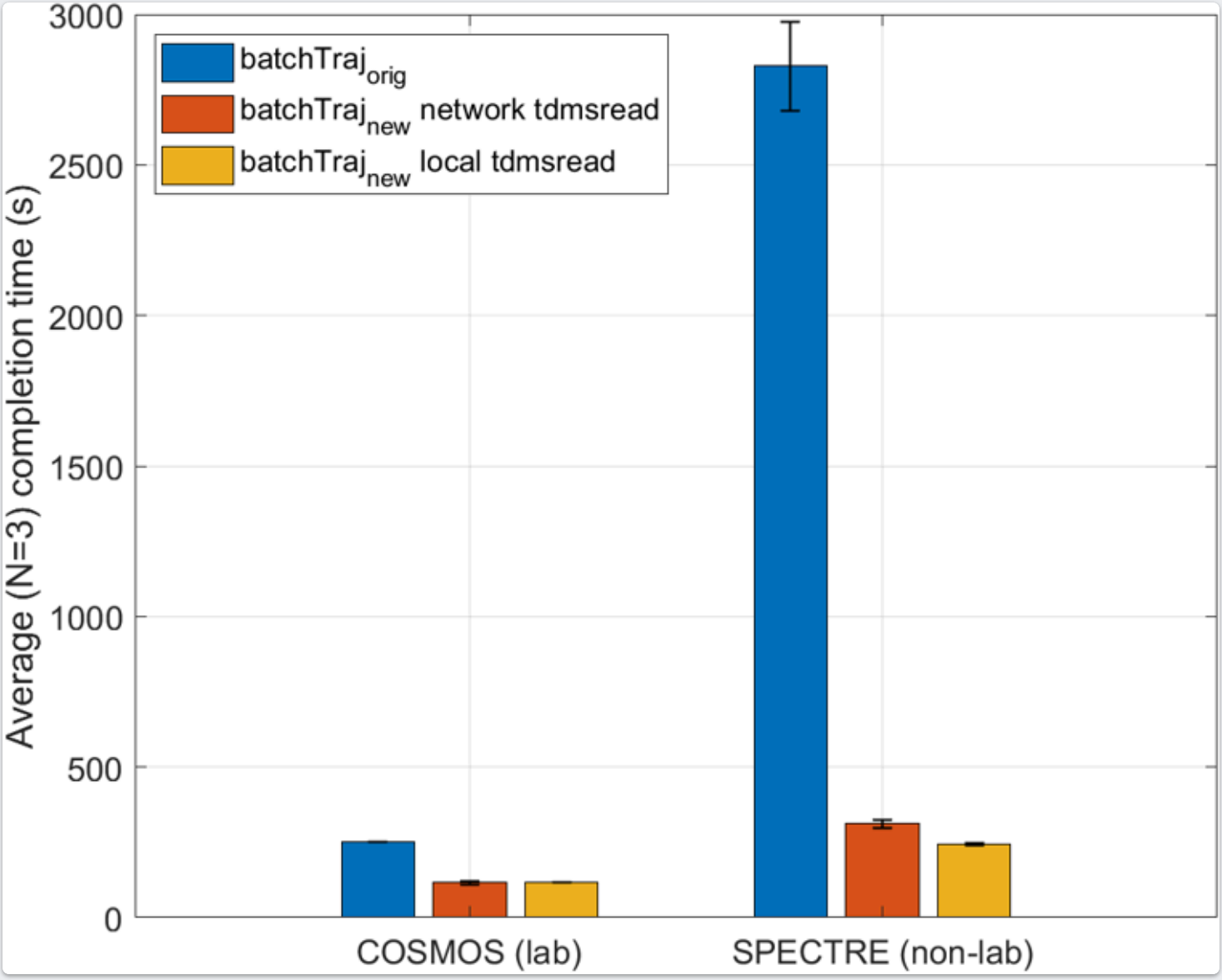
For reference, here are the specs of the two PCs used for this benchmark:

	COSMOS (lab)	SPECTRE (non-lab)
CPU	Intel i7-7820X (8 cores, 16 threads)	Intel Ultra 7 155H (16 cores, 22 logic threads)
GPU	NVIDIA GeForce GTX 1050 Ti	Intel Arc
RAM	64 GB	32 GB
Access to data2	Ethernet	Network shared folder via DukeBlue
Window version	Win 10	Win 11
MATLAB version	2024b	2024a

To test the processing time, I used `tic` and `toc` function to get the elapsed time as both scripts go through a batch of 5 trajectories, each with duration of ~1 minute.

The elapsed times were recorded in triplicates, and the average time was used in the plot below. For COSMOS, the speed improved 2x when using this script. For SPECTRE, a massive 10x improvement was

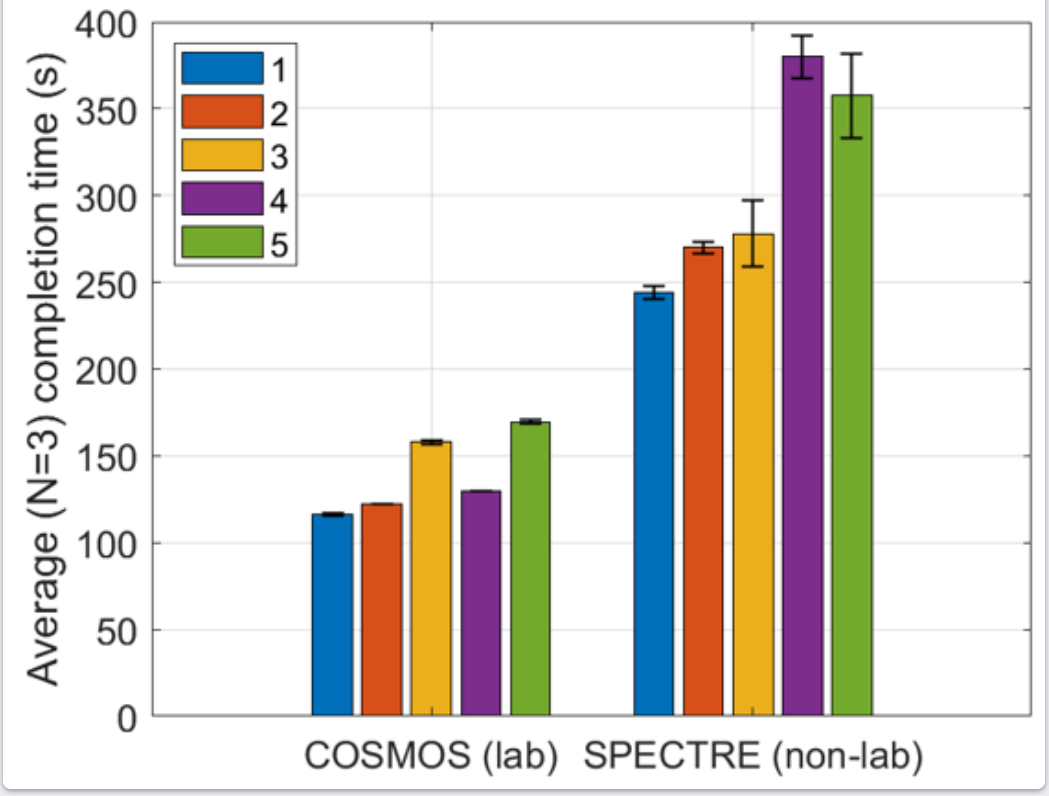
achieved.



Performance comparison between the original batchTraj and newBatchTraj (this script). Within newBatchTraj, compare between reading the TDMS directly from network or from a local TDMS copy.

For lab PC, it makes no difference whether you read the TDMS files locally or from network. However, on non-lab PC, the speed improve by ~20% when the local TDMS copy was used; thus this approach is highly recommended.

ID	Params		
	createDataCollage	saveCollagePNG	calc1dMSD
1	☐	☐	☐
2	☑	☐	☐
3	☑	☑	☐
4	☑	☐	☑
5	☑	☑	☑



Benchmark for when running additional features in this script. In all cases, TDMS files are read locally.